

Konfiguration der Softwareentwicklung und Qualitätsplanung

Vanessa Künzel & Jasmin Hundt

Konfiguration der Softwareentwicklung

Anforderungsmanagement

Artefakt	Beschreibung	Verantwortlich
Anforderungsdokument (s.Liefergegenstand Projektkonfiguration)	Vollständige Beschreibung aller funktionalen und nicht-funktionalen Anforderungen	Jasmin
User Stories	Anforderungen aus Nutzerperspektive	Vanessa
Use Case Beschreibungen	Detaillierte Szenarien für jeden Anwendungsfall	Vanessa
Product Backlog	Priorisierte Liste aller Anforderungen als GitHub Issues	Jasmin

Die Anforderungserhebung erfolgt durch Analyse der Aufgabenstellung sowie Teamgespräche. Anforderungen werden als User Stories und Use Cases beschrieben. Technische Aufgaben werden als GitHub Issues verwaltet, während Liefergegenstände und Meilensteine in Redmine gepflegt werden.

Zur Qualitätssicherung überprüfen beide Teammitglieder jede Anforderung, bevor sie ins Backlog aufgenommen werden. Dabei wird die Vollständigkeit und Konsistenz mit der Aufgabenstellung geprüft sowie sichergestellt, dass jede Anforderung mindestens einen konkreten Testfall ermöglicht.

Artefakte und Zwischenergebnisse

Artefakt	Format	Ablageort	Verantwortlicher	Meilenstein
Anforderungsliste	Dokument	Redmine	Jasmin	MS2
Architekturbeschreibung	Dokument	Redmine	Jasmin	MS2
Datenmodell	Diagramm	GitHub	Jasmin	MS3
Wireframes	Skizzen	GitHub	Vanessa	MS2

Product Backlog	Issues	GitHub	Jasmin	Laufend
Sprint Backlog	Issues	GitHub	Jasmin	Wöchentlich
Implementierter Prototyp	Quellcode	GitHub	Beide	MS3/4
Bugliste	Issues(bug)	GitHub	Beide	Laufend
Testfälle	Dokument	GitHub	Beide	MS3/4
Benutzerhandbuch	Dokument	GitHub	Beide	MS4
Projektdokumentation	Dokument	TuritIn	Beide	MS5

Eingesetzte Programmiersprachen

Bereich	Sprache	Begründung
Frontend	TypeScript	Typsicherheit, bessere IDE-Unterstützung gegenüber JavaScript
	HTML/CSS	Standard für Weboberflächen
Backend	TypeScript	Leichtgewichtige Serverseite, kompatibel mit Express
Datenbankschema	Prisma Schema Language	Deklarative Datenbankdefinition, typsicherer ORM-Zugriff
Konfiguration	YAML, JSON, Shell	Docker-Compose, npm-Scripts, Umgebungskonfiguration

Eingesetzte Systeme

Zweck	System/Werkzeug	Version/URL	Bemerkung
Versionsverwaltung	GitHub	http://github.com	Zentrales Repository für Quellcode und Dokumentation
Liefergegenstand Verwaltung	Redmine	Redmine-Projekt	Meilensteinverfolgung, Liefergegenstände
Ergänzende Aufgaben	GitHub Issues	http://github.com	Backlog Items, Bugs
Entwicklung	VS Code	aktuell	Primäre Entwicklungsumgebung

Frontend-Framework	React + Vite	React 19, Vite 8	Frontend Entwicklung
Backend-Framework	Express.js	4.x	REST-API-Server
ORM	Prisma	5.x	Typsicherer Datenbankzugriff
Datenbank	SQLite	via Prisma	Lokale, dateibasierte Datenbank
Containerisierung	Docker Compose	Docker 20+	Konsistente Entwicklungs- und Laufzeitumgebung
UI-Design	Händische Skizzen	-	Wireframes
Styling	Bootstrap	5.x	Responsive UI-Komponenten
Kommunikation	Teams	aktuell	Wöchentliche Sprint Meetings
Dokumentation	Google Docs	aktuell	Erstellung der Liefergegenstände als PDF

Rollen und Ergebnisverantwortung

Rolle	Person	Aufgaben	Ergebnisverantwortung
Projektleitung	Jasmin	Projektplanung, Meilensteinverfolgung, Koordination	Einhaltung des Meilensteinplans, Redmine-Pflege
Frontend Entwicklung	Vanessa	Umsetzung der React Oberfläche, UI-Komponenten	Getestete Frontend Komponenten
Backend Entwicklung	Jasmin	Umsetzung der REST-API, Datenbankmodell	Lauffähige, getestete Backend-Komponenten
Architektur	Beide	Entwurf Systemarchitektur, API-Design, Datenmodell	Architekturbeschreibung und Datenmodell
Testing	Beide	Entwurf und Durchführung von Tests	Testprotokolle
Code Reviews	Beide	Gegenseitige Review vor Merge auf main	Codequalität
Dokumentation	Beide	Erstellung aller	Vollständigkeit

		Liefergegenstände	
--	--	-------------------	--

Vorgehensmodell

Das Projekt wird nach einem agilen Vorgehensmodell in Anlehnung an Scrum durchgeführt, angepasst an die Rahmenbedingungen eines Zweipersonen-Studienprojekts.

- Sprint-Länge: 1 Woche

Konfiguration der Liefergegenstände

1. **Projektkonfiguration** (MS 1)
Umfang: ca. 7 Seiten
Inhalt: Projektvision, Teamvorstellung, Anforderungsüberblick, Lösungsansatz, Annahmen/Beschränkungen, Meilensteinplan, Aufwandsschätzung, eingesetzte Systeme, Infrastruktur, Risikomanagement
2. **Projektvideo** (MS 2)
Umfang: ca. 5 min Präsentation
Inhalt: Teamvorstellung, Projektvorstellung
3. **Konfiguration der Softwareentwicklung und Qualitätsplanung** (MS 3)
Umfang: ca. 5 Seiten
Inhalt: Konfiguration der Softwareentwicklung, Konfiguration der Liefergegenstände, Qualitätsplanung
4. **Implementierter Prototyp** (MS 3 / MS 4)
Umfang: vollständiger Quellcode aller Komponenten
Inhalt: Frontend (React/TypeScript), Backend (Node.js/Express/Prisma), Datenbank (SQLite-Schema und Migrationen), docker-compose.yml, README mit Inbetriebnahme-Anleitung
5. **Testdokumentation** (MS 4)
Umfang: ca. 5-8 Seiten
Inhalt: Teststrategie, Testfälle, Testprotokoll mit Ergebnissen, Fehlerliste
6. **Benutzerhandbuch** (MS 5)
Umfang: ca. 5–10 Seiten
Inhalt: Einleitung, Installation und Start, Lernziele verwalten, Lernzeiten planen, Timer und Zeiterfassung, Dashboard und Auswertungen, Erinnerungen, häufige Probleme (FAQ)
7. **Projektbericht** (MS 6)
Umfang: ca. 15–25 Seiten
Inhalt: Projektüberblick, Projektverlauf und Meilensteine, Soll-Ist-Vergleich (Aufwand und Termine), Ergebnisse und Liefergegenständen, technische Erkenntnisse, Teamreflexion, Ausblick
8. **Abschlusspräsentation** (MS 5)
Umfang: ca. 10–15 Folien, 15–20 Minuten
Inhalt: Projektvorstellung, Anforderungen und Architektur, Live-Demo, Qualitätssicherung und Tests, Projektverlauf und Learnings

Qualitätsplanung

Qualitätsziele

Qualitätsziel	Metrik	Zielwert	Rolle
Funktionalität: Alle definierten Anwendungsfälle sind implementiert	Abnahmetests gegen User Stories	100% der Must-Have-Anforderungen	Beide
Benutzerfreundlichkeit: Intuitive Bedienung ohne Einarbeitung	Manuelle Nutzertests	Keine kritischen Mängel	Vanessa
Performance: Antwortzeiten unter 2 Sekunden	Manuelle Messung mit Browser DevTools	<2 Sekunden für alle Standardoperationen	Jasmin
Responsivität: Nutzbar auf Desktop und Mobilgeräten	Manuelles Testen in Chrome DevTools	Keine Layout-Brüche	Vanessa
Wartbarkeit: Verständlicher, dokumentierter Code	Code Reviews	Verständliche Implementierung	Beide
Korrektheit: Zeiterfassung rechnet fehlerfrei	Tests für Timer-Logik und Zeitberechnung	100% Test-Pass	Jasmin

Qualität der Liefergegenstände

Alle Liefergegenstände werden anhand folgender Kriterien geprüft, bevor sie in Redmine eingereicht werden:

- Vollständigkeit: Alle geforderten Abschnitte sind vorhanden
- Korrektheit: Inhaltliche Aussagen sind korrekt und konsistent
- Verständlichkeit: Dokument ist ohne Rückfragen verständlich
- Formatierung: Einheitliches Format
- Rechtschreibung & Grammatik: Dokument ist auf Fehler geprüft
- Abgabe: Dokument ist fristgerecht in Redmine hochgeladen